LAB1 : Implementing Docker and understanding rest , gRPC,MQTT Communication Protocol

Name : Divyansh Jaiswal
Roll No.: 20231130
Course : AI in 5g (CS342)

Objective

The objective of this experiment is to implement and demonstrate REST API, gRPC, and MQTT based communication using Docker containers on Ubuntu Linux. The lab focuses on containerization, inter-service communication, and protocol comparison.

First this : Docker Version Verification

```
codehashira23@codehashira23:~/AI5G_Lab1$ docker --version
Docker version 29.2.0, build 0b9d198
```

Command Executed: `docker --version`
Output Obtained: Displays the installed Docker version
Concept Demonstrated: Verifies that Docker Engine is successfully installed and available on the system

```
codehashira23@codehashira23:~/AI5G_Lab1$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Command Executed: `docker run --rm hello-world`
Output Obtained: Confirmation message from Docker
Concept Demonstrated: Confirms Docker can pull images from Docker Hub and run containers correctly

```
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$  docker build -t rest-advanced .
```

```
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$  docker build -t rest-advanced .
docker run -d --name rest-advanced -p 5000:5000 rest-advanced
[+] Building 10.2s (10/10) FINISHED                                                                                    dock
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 256B
 => [internal] load metadata for docker.io/library/python:3.10-slim
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.10-slim@sha256:f5d029fe39146b08200bcc73595795ac19b85997ad0e5001a02c7c32e8769efa
 => => resolve docker.io/library/python:3.10-slim@sha256:f5d029fe39146b08200bcc73595795ac19b85997ad0e5001a02c7c32e8769efa
 => [internal] load build context
 => => transferring context: 1.34kB
 => CACHED [2/5] WORKDIR /app
 => [3/5] COPY app/requirements.txt .
 => [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY app/ .
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:7e638ec4d752502376082b1a05559cc472ed1ed232649e2f1013aa258c7ef805
 => => exporting config sha256:c28f960c0fe5d07fb5e17c459bab9f00ece41c8fb2e985d6008bc0e0615c9d8b
 => => exporting attestation manifest sha256:5aa0802fd3598e6ec717e903953c94e9929d3e70ad6456b96bc78011b3baa5ff
 => => exporting manifest list sha256:08a8874f3dcfc80a7739505246051625 00d87402fa6892340663514ad257e412
 => => naming to docker.io/library/rest-advanced:latest
 => => unpacking to docker.io/library/rest-advanced:latest
f01dde31c253960b46a5494d60a0e02c8bee7b828856c2a233ec864386c6bfc6
```

Command Executed: `docker build -t rest-advanced .`
Output Obtained: Docker image built successfully
Concept Demonstrated: Containerization of a RESTful Flask application using a Dockerfile

```
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$ docker ps
CONTAINER ID   IMAGE           COMMAND            CREATED         STATUS         PORTS                                       NAMES
957d50ba07db   rest-advanced   "python main.py"   8 seconds ago   Up 8 seconds   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp   rest-adv
anced
```

Command Executed: `docker ps`
Output Obtained: REST container running and listening on port 5000
Concept Demonstrated: Successful execution of a Dockerized REST service with port mapping

```
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$ curl http://localhost:5000/metrics
{"requests_total":1}
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$ curl http://localhost:5000/health
{"status":"healthy"}
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$ curl http://localhost:5000/api/v1/data
{"sensor":"heart_rate","unit":"bpm","value":72}
```

Command Executed: `curl http://localhost:5000/health`
Output Obtained: JSON response indicating service health
Concept Demonstrated: REST GET request over HTTP using a Dockerized Flask application

Command Executed: `curl http://localhost:5000/metrics`
Output Obtained: JSON metrics data

Concept Demonstrated: Stateless REST endpoint providing service metrics

Command Executed: `curl http://localhost:5000/api/v1/data`
Output Obtained: Sensor data in JSON format
Concept Demonstrated: RESTful data retrieval using HTTP GET method

Command Executed: HTTP POST request using curl
Output Obtained: Server acknowledgment with received JSON payload
Concept Demonstrated: RESTful data submission using HTTP POST and JSON payloads

Summary:
This experiment demonstrates the implementation of a RESTful web service using Flask and Docker. The service exposes multiple HTTP endpoints supporting GET and POST methods with JSON-based communication. Docker provides isolation, portability, and ease of deployment, while REST enables stateless client–server communication over HTTP.

==========

Part 2 : gRPC using Docker

```
codehashira23@codehashira23:~/AI5G_Lab1/rest-service$ curl -X POST http://localhost:5000/api/v1/data \
-H "Content-Type: application/json" \
-d '{"sensor":"temperature","value":36.5}'
{"data":{"sensor":"temperature","value":36.5},"message":"Data received"}
```

```
codehashira23@codehashira23:~$ cd AI5G_Lab1
codehashira23@codehashira23:~/AI5G_Lab1$ mkdir grpc-lab
cd grpc-lab
mkdir proto server client
codehashira23@codehashira23:~/AI5G_Lab1/grpc-lab$ nano proto/hello.proto
codehashira23@codehashira23:~/AI5G_Lab1/grpc-lab$ pip3 install grpcio grpcio-tools --break-system-packages

Usage:
  pip3 install [options] <requirement specifier> [package-index-options] ...
  pip3 install [options] -r <requirements file> [package-index-options] ...
  pip3 install [options] [-e] <vcs project url> ...
  pip3 install [options] [-e] <local project path> ...
  pip3 install [options] <archive url/path> ...

no such option: --break-system-packages
```

making folders and installlingthe required tools

```
codehashira23@codehashira23:~/AI5G_Lab1/grpc-lab$ docker compose down --remove-orphans
docker system prune -f
docker compose up --build
WARN[0000] /home/codehashira23/AI5G_Lab1/grpc-lab/docker-compose.yml: the attribute `version` is obsolete, it
emove it to avoid potential confusion
[+] down 3/3
 ✔ Container grpc-lab-client-1 Removed
 ✔ Container grpc-lab-server-1 Removed
 ✔ Network grpc-lab_default    Removed
Deleted build cache objects:
n1afbhyqc4d16u3gu8pog04my
f25s5mbztjtjrxkbrxky12me9
lue6y5j0swsa3rc4ng4uubb48
sjs1c3am3v1y4t65ohdtkvram
rl3nvezk09y1w7q9ixhamj88j
```

output :

```
[+] up 5/5
 ✔ Image grpc-lab-server       Built
 ✔ Image grpc-lab-client       Built
 ✔ Network grpc-lab_default    Created
 ✔ Container grpc-lab-server-1 Created
 ✔ Container grpc-lab-client-1 Created
Attaching to client-1, server-1
client-1  | Server Response: Hello Ubuntu LAB, from gRPC Server!
client-1 exited with code 0
```

- **Command Executed:** `docker compose up --build`
  - **Output Obtained:** Client and server containers built
  - **Concept Demonstrated:** Multi-container orchestration

==========

Part 3 : MQTT Using Docker

```
codehashira23@codehashira23:~$ cd ~/AI5G_Lab1
mkdir mqtt-lab
cd mqtt-lab
```

Command Executed: `mkdir mqtt-lab`

- **Concept Demonstrated:** Project setup for MQTT experiment

```
codehashira23@codehashira23:~/AI5G_Lab1/mqtt-lab$ nano docker-compose.yml
codehashira23@codehashira23:~/AI5G_Lab1/mqtt-lab$ docker compose up -d
[+] up 8/8
 ✔ Image eclipse-mosquitto:2 Pulled                                                                    19.7s
 ✔ Network mqtt-lab_default  Created                                                                    0.1s
 ✔ Container mqtt-broker     Created                                                                    0.2s
codehashira23@codehashira23:~/AI5G_Lab1/mqtt-lab$ docker ps
CONTAINER ID   IMAGE                  COMMAND                  CREATED             STATUS             PORTS
           NAMES
3618cd8fd364   eclipse-mosquitto:2    "/docker-entrypoint.…"   3 seconds ago       Up 2 seconds       0.0.0.0:1883->1883/tcp, [::]:1883-
>1883/tcp   mqtt-broker
957d50ba07db   rest-advanced          "python main.py"         About an hour ago   Up About an hour   0.0.0.0:5000->5000/tcp, [::]:5000-
>5000/tcp   rest-advanced
```

Concept Demonstrated: MQTT broker configuration using Docker
Command Executed: `docker compose up -d`

- **Output Obtained:** Mosquitto broker running
- **Concept Demonstrated:** MQTT broker deployment

```
codehashira23@codehashira23:~$ docker exec -it mqtt-broker mosquitto_pub -t ai5g/topic -m "Hello from MQTT Publisher"
codehashira23@codehashira23:~$
```

- **Command Executed:** `mosquitto_pub  (in terminal 2)`
- **Output Obtained:** Message delivered to subscriber
- **Concept Demonstrated:** MQTT publish–subscribe communication

```
codehashira23@codehashira23:~/AI5G_Lab1/mqtt-lab$ docker exec -it mqtt-broker mosquitto_sub -t ai5g/topic
Hello from MQTT Publisher
```

Command Executed: `mosquitto_sub`

- **Concept Demonstrated:** MQTT subscriber listening on topic

Conclusion

In this lab, Docker was used to containerize and deploy different communication models, namely REST, gRPC, and MQTT, on an Ubuntu Linux system. A REST API was implemented and tested using HTTP requests to demonstrate stateless client–server communication. gRPC was implemented using Protocol Buffers to enable efficient binary communication between a client and a server running in separate Docker containers. MQTT communication was demonstrated using a Mosquitto broker to showcase lightweight publish–subscribe messaging.

Through this experiment, the concepts of containerization, microservices, and inter-service communication were clearly understood. The lab highlighted the practical differences between REST, gRPC, and MQTT in terms of communication style, efficiency, and use cases. Overall, the experiment successfully demonstrated how Docker simplifies deployment and testing of modern network communication protocols.